

Thread Usage

Definition

A **thread** is a thread of control that runs inside a process. Unlike separate processes, multiple threads share the same address space and data, but each thread has its own execution flow.

1. Basic Idea

Traditional Process Model

In a traditional operating system, each process has:

- its own address space,
- and one thread of control.

What Threads Add

Threads allow multiple threads of control to exist inside the **same** address space. They run in quasi-parallel and behave almost like separate processes, except that memory is shared.

2. Why Threads Are Useful

Main Reasons

Threads are useful for four main reasons:

1. They simplify programs in which multiple activities happen at the same time.
2. They are lighter weight and faster to create and destroy than processes.
3. They improve performance when computation and I/O can overlap.
4. They allow real parallelism on multiprocessor systems.

Reason	Meaning
Simpler design	Different activities can be written as separate sequential threads instead of one complicated interrupt-driven program.
Lower overhead	Thread creation and destruction are often much faster than process creation and destruction.
Better overlap	One thread can compute while another waits for I/O.
Parallel hardware use	On multi-CPU systems, different threads can run truly in parallel.

Why Processes Alone Are Not Enough

Multiple processes cannot replace threads in some applications because separate processes do **not** share the same address space by default. Threads do.

3. Example 1: Word Processor

Two-Thread Design

Suppose a word processor has:

- one thread for user interaction,
- one background thread for reformatting the document.

Why This Helps

If the user edits page 1 of a very large document, the formatting thread can reformat the book in the background while the interactive thread still responds to scrolling, typing, and mouse actions.

Three-Thread Design

A third thread can periodically save the document to disk for backup, without freezing the interface or interfering with reformatting.

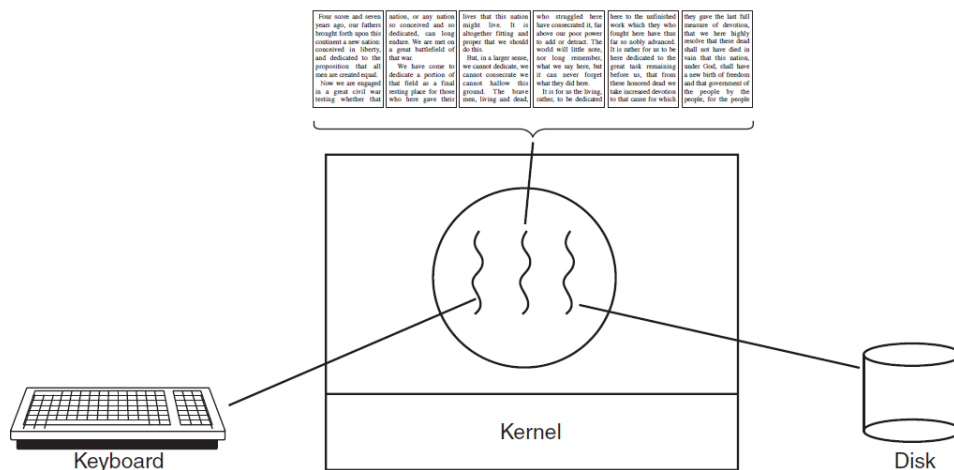


Figure 1: A word processor with three threads.

Why Three Processes Would Fail

Using three separate processes would be inappropriate because all three activities need access to the same document in memory. Threads work because they share that memory.

4. Example 2: Spreadsheet

Spreadsheet Use Case

When a user changes one cell, many other cells may need to be recomputed. A background thread can perform recalculation while the interactive thread still accepts new user actions.

Backup Thread Again

As with a word processor, a third thread can perform periodic disk backups independently.

5. Example 3: Multithreaded Web Server

Server Structure

A common Web server design uses:

- one **dispatcher thread** to receive requests,
- multiple **worker threads** to process them,
- and a shared **Web page cache**.

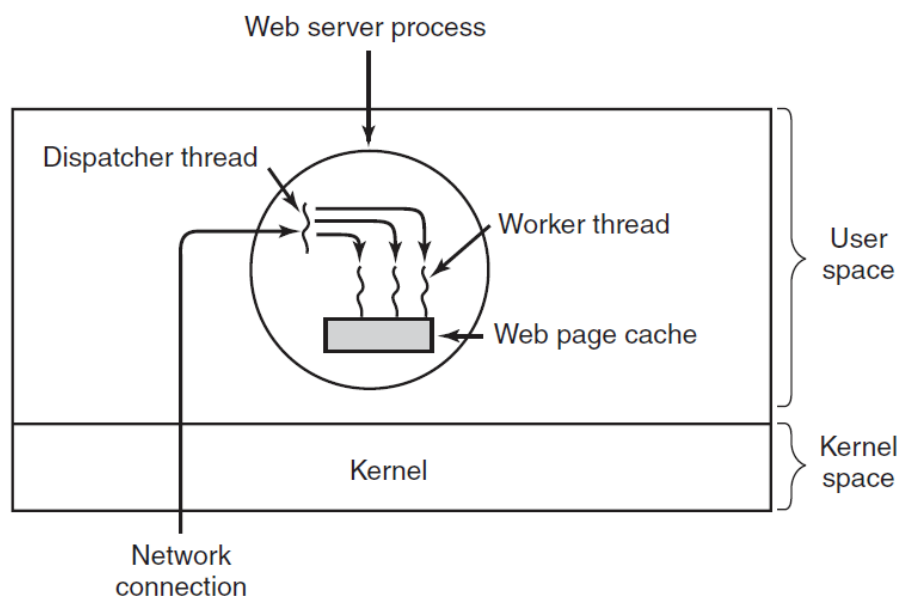


Figure 2: A multithreaded Web server.

How It Works

The dispatcher accepts a request, chooses an idle worker thread, passes the request to it, and wakes it up. The worker checks the shared cache. If the page is missing, it reads from disk and blocks while the disk I/O is in progress.

Performance Benefit

While one worker is blocked on disk I/O, another thread can run. This keeps the server responsive and increases the number of requests handled per second.

5.1 Dispatcher and Worker Outline

Dispatcher Thread Pseudocode

```
while (TRUE) {
    get_next_request(&buf);
    handoff_work(&buf);
}
```

Worker Thread Pseudocode

```
while (TRUE) {
    wait_for_work(&buf);
    look_for_page_in_cache(&buf, &page);
    if (page_not_in_cache(&page)) {
        read_page_from_disk(&buf, &page);
    }
    return_page(&page);
}
```

Programming Advantage

Each thread can still be written as a simple sequential program, even though the whole server behaves concurrently.

6. Without Threads

Single-Threaded Server

If the Web server has only one thread, then while it waits for disk I/O it cannot process any other request. This wastes CPU time and reduces throughput.

Finite-State Machine Approach

Without threads, one alternative is to use nonblocking I/O and explicitly save the state of each request in a table. This design is called a **finite-state machine**.

Why It Is Harder

In a finite-state machine, the programmer must explicitly save and restore computation state whenever the server switches from one request to another. This is much harder than writing blocking sequential code.

Model	Characteristics
Threads	Parallelism with blocking system calls
Single-threaded process	No parallelism, but still uses blocking system calls
Finite-state machine	Parallelism with nonblocking system calls and interrupts

7. Example 4: Large Data Processing

Three-Thread Pipeline

An application that processes very large amounts of data can be organized with:

- an input thread,
- a processing thread,
- an output thread.

How the Pipeline Works

The input thread fills an input buffer. The processing thread reads from that buffer, processes the data, and writes results to an output buffer. The output thread writes those results to disk.

Key Requirement

This works properly only if a blocking system call blocks the **calling thread**, not the entire process.

8. Main Conclusion

Takeaway

Threads make it possible to keep the simple sequential programming style of blocking calls while still achieving concurrency and better performance. Their biggest advantage is shared memory plus lighter-weight control flows.

9. Quick Review

Exam-Focused Points

- A thread is a thread of control inside a process.
- Multiple threads share the same address space and data.
- Threads are useful when many activities happen at once.
- Threads are lighter weight than processes and are usually faster to create and destroy.
- Threads improve performance when CPU work and I/O can overlap.
- A multithreaded Web server can use a dispatcher thread and several worker threads.
- Threads keep sequential programming simple while still allowing concurrency.
- A finite-state machine can also achieve concurrency, but it is harder to program.
- Threads are ideal when different activities need access to the same shared data.